

MOVIE REVIEW SENTIMENT ANALYSIS

Complete Project Architecture, Theoretical Compendium & Interview Master Handbook

Role Target: Graduate Engineer Trainee (GET) – AI/ML

Focus Area: Natural Language Processing (NLP) & Classical Machine Learning

Benchmark Dataset: 50,000 IMDB Large Movie Review Dataset (Balanced)

Winning Architecture: TF-IDF (Unigram + Bigram) + Logistic Regression

Model Generalization: Accuracy: 90.23% | F1-Score: 90.38% | Latency: < 2.5 ms

GitHub Repository: github.com/SoumyaRanjan68/Movie-sentiment

Project Architecture & System Specifications

⚡ Sentiment Intelligence & Text Analytics Platform

Production-Grade Natural Language Processing & Machine Learning Classification Engine

Python 3.13 scikit-learn 1.7.0 Streamlit 1.63.0 Latency <5ms Accuracy 90.23%

An end-to-end, high-performance, and fully explainable Natural Language Processing (NLP) system engineered for binary sentiment classification (*Positive* vs. *Negative*).

Designed with modular object-oriented architecture, rigorous cross-validation to prevent data leakage, calibrated probabilistic inference, and an interactive analytics web platform.

🚀 System Highlights & Engineering Rationale

Key Attribute	Specification	Engineering Justification
Model Architecture	L2-Regularized Logistic Regression	Convex optimization with guaranteed convergence, highly suited for sparse text feature spaces.
Feature Extraction	TF-IDF (Unigrams + Bigrams)	Captures contextual phrases and negations without exploding dimensionality.
Vocabulary Size	10,000 Informative N-grams	Optimizes memory consumption while capturing >95% of informative variance.
Inference Latency	< 2.5 ms (CPU-based)	Zero GPU requirements; ideal for high-throughput microservices.
Test Accuracy	90.23% (Holdout Test Set)	Strong generalization on 9,917 unseen test samples.
Test F1-Score	90.38% (Macro Avg: 0.90)	Symmetric precision and recall across both sentiment classes.

Processing Pipeline & Architecture

Raw Unstructured Text Corpus (50,000 Records)

|



[Stage 1: Deterministic Text Preprocessing]

- HTML entity unescaping & markup tag excision (
, <p>)
- URL removal (http/https/www)
- Contraction expansion (e.g. "didn't" -> "did not" to preserve negation signals)
- Case normalization & alphanumeric symbol filtering

|



[Stage 2: Exploratory Data Analysis (EDA)]

- Target class balance verification (50.2% Positive : 49.8% Negative)
- Document length & word count distributions

|



[Stage 3: Stratified Train / Test Partitioning]

- 80% Training (39,665 samples) / 20% Holdout Test (9,917 samples)
- Strict Data Leakage isolation: Feature matrices fitted exclusively on train

|



[Stage 4: High-Dimensional TF-IDF Feature Extraction]

- Unigram & Bigram combinations (ngram_range=(1, 2))
- Sublinear term frequency scaling: $1 + \log(\text{tf})$
- Noise reduction: minimum document frequency (min_df=3)

|



[Stage 5: Model Training & Benchmarking]

- Model A: Logistic Regression (C=1.0, L-BFGS solver)
- Model B: Multinomial Naive Bayes (Laplace smoothing alpha=1.0)

|



[Stage 6: Diagnostics, Metrics & Model Serialization]

- Accuracy, Precision, Recall, F1-Score, and Confusion Matrix Heatmaps
- Model checkpoints & vectorizer serialized via Joblib

|



[Stage 7: Interactive Streamlit Platform & Inference Engine]

- Real-time classification, confidence gauges, and preprocessed text inspection

Repository Structure

```
movie-sentiment-analysis/
├── data/
│   ├── raw/
│   │   └── imdb_reviews.csv          # Benchmark raw dataset (50k records)
│   └── processed/
│       └── cleaned_reviews.csv      # Deduplicated & normalized records (49,582 clean)
├── notebooks/
│   └── eda_and_modeling.ipynb      # Jupyter notebook for exploratory data science
├── src/
│   ├── __init__.py                 # Package initialization
│   ├── data_preprocessing.py        # Preprocessing pipeline, regex rules & contraction map
│   ├── train_model.py               # Stratified split, TF-IDF vectorization & model training
│   ├── evaluate_model.py            # Diagnostic visual analytics & confusion matrix generation
│   └── predict.py                   # Self-contained SentimentPredictor inference engine
├── models/
│   ├── best_model.joblib            # Serialized Logistic Regression production weights
│   ├── tfidf_vectorizer.joblib      # Serialized 10,000-term TF-IDF vectorizer
│   ├── logistic_regression_model.joblib # LR model checkpoint
│   ├── naive_bayes_model.joblib     # MNB model checkpoint
│   └── model_metadata.json          # Pipeline configuration & performance metadata
├── reports/
│   └── figures/
│       ├── class_distribution.png    # Target class balance plot
│       ├── review_length_distribution.png # Word count density by sentiment
│       ├── confusion_matrix_logistic_regression.png
│       ├── confusion_matrix_naive_bayes.png
│       └── model_comparison_metrics.png # Comparative KPI bar chart
├── app.py                           # Production Streamlit web application
├── requirements.txt                  # Project dependencies
├── README.md                        # Comprehensive system documentation
├── INTERVIEW_NOTES.md               # In-depth technical handbook & Q&A guide
└── .gitignore                       # Environment & cache ignore rules
```

Empirical Evaluation & Benchmark Results

Both classifiers were trained on **39,665 samples** and evaluated on **9,917 holdout test samples**:

Model	Accuracy	Precision	Recall	F1-Score	Inference Latency
Logistic Regression (Selected Best)	90.23%	89.32%	91.46%	90.38%	~2.1 ms
Multinomial Naive Bayes	86.95%	85.97%	88.43%	87.18%	~1.4 ms

Confusion Matrix Breakdown (Holdout Test Set: 9,917 Samples)

- **True Negatives (TN):** 4,395
- **False Positives (FP - Type I Error):** 545
- **False Negatives (FN - Type II Error):** 425
- **True Positives (TP):** 4,552
- **Diagnostic Finding:** The error profile is symmetrical across positive and negative sentiments, showing no directional bias.

Installation & Local Execution

1. Prerequisites & Environment Setup

```
# Clone the repository
git clone <repository-url>
cd movie-sentiment-analysis

# Create and activate virtual environment
python -m venv venv
venv\Scripts\activate      # Windows
# source venv/bin/activate # macOS / Linux

# Install dependencies
pip install -r requirements.txt
```

2. Execute Training & Diagnostic Pipeline

```
# Run data preprocessing and model training
python src/train_model.py

# Generate statistical figures and confusion matrix heatmaps
python src/evaluate_model.py
```

3. Programmatic Inference CLI

```
python src/predict.py
```

4. Launch the Interactive Web Application

```
streamlit run app.py --server.port 8502
```

Access the interface via `http://localhost:8502`.

Python API Usage Example

```
from src.predict import SentimentPredictor

# Initialize predictor (loads cached model and vectorizer)
predictor = SentimentPredictor(models_dir='models')

# Execute inference
text = "The screenplay was brilliant and the visual effects were extraordinary."
result = predictor.predict(text)

print(result)
# Output:
# {
#   'prediction': 'Positive',
#   'confidence': 0.942,
#   'positive_probability': 0.942,
#   'negative_probability': 0.058,
#   'cleaned_text': 'the screenplay was brilliant and the visual effects were extraordinary'
# }
```

Production Deployment & Scalability Considerations

- **Containerization:** Easily packageable via Docker using an official Python slim base image (`python:3.13-slim`).
- **REST API Microservice:** Can be exposed via **FastAPI** with `uvicorn` workers for high-concurrency throughput.
- **Monitoring:** Real-time logging of prediction confidence allows proactive detection of data drift and vocabulary distribution shifts.



AI/ML INTERVIEW MASTER HANDBOOK: MOVIE REVIEW SENTIMENT ANALYSIS

**Professional Engineering Guide & Technical Architecture Compendium Handbook.
Master these concepts and defend your project with absolute confidence!***